	BANNARI AMMAN INSTITUTE OF TECHNOLOGY SATHYAMANGALAM – 638 401 DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING LAB MANUAL	

22EC603

22EC603 - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

S6 -ECE

Year of Release: 2025

	BANNARI AMMAN INSTITUTE OF TECHNOLOGY **SATHYAMANGALAM – 638 401** DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING LABORATORY MANUAL	

22EC603

22EC603 - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

S6 -ECE

Approved by
Dr. D. SASIKALA
HOD - CSE

Year of Release: 2025

2022: 22EC603: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

List of Experiments

1. Develop PEAS Description for given AI task i) Autonomous self-driving car ii) Online Shopping iii) Personal AI Assistant iv) Medical Diagnosis System v) AI based spam email filter, vi) Robot Vacuum Cleaner
2. Design of an AI agent for 2d grid maze navigation using heuristic search strategies
3. Build and train machine learning classifier to categorize sales data using decision tree classifier
4. Create and train a linear regression model for predicting house prices
5. Design and implement of a reinforcement learning based treasure collection agent in a 6*6 grid environment
6. Design and implement NLP based naive bayes classifier for classification of spam email detection.

DEVELOP PEAS DESCRIPTION FOR GIVEN AI TASK I) AUTONOMOUS SELF DRIVING CAR II) ONLINE SHOPPING III) PERSONAL AI ASSISTANT IV) MEDICAL DIAGNOSIS SYSTEM V) AI BASED SPAM EMAIL FILTER VI) ROBOT VACUUM CLEANER

Exp. No:1

Date :

AIM:

The AIM is to develop PEAS Description for given AI task i) Autonomous self-driving car ii) Online Shopping iii) Personal AI Assistant iv) Medical Diagnosis System v) AI based spam email filter vi) Robot Vacuum Cleaner

ALGORITHM:

- Parse HTML to structure the resumed content into sections.
- Use CSS to apply visual styles to sections.
- Implement a vertical timeline for experience and education using icons and lines for separation.
- Adopt layout responsive with CSS media queries for various devices.
- Ensure text readability and visual appeal across screen sizes.
- Highlight key information through strategic styling and layout adjustments.

DESCRIPTION:

PEAS is a framework used to define and describe an intelligent agent in artificial intelligence. It stands for:

- **P – Performance Measure**: The criteria by which the success of the agent's behavior is judged.
- **E – Environment**: The external context or surroundings in which the agent operates.
- **A – Actuators**: The components that allow the agent to take actions and affect the environment.
- **S – Sensors**: The components that allow the agent to perceive or receive information from the environment

i) Autonomous Self-Driving Car

Performance Measure:
Safe and efficient driving, obeying traffic laws, avoiding collisions, passenger comfort, minimal fuel consumption.

Environment:
Roads, traffic lights, pedestrians, other vehicles, road signs, weather conditions.

Actuators:
Steering wheel, accelerator, brake, horn, signal indicators.

Sensors:
Cameras, LiDAR, radar, GPS, ultrasonic sensors, speedometer.

---

ii) Online Shopping System (Recommendation Engine)

Performance Measure:
Accuracy of product recommendations, increased user engagement, sales growth, customer satisfaction.

Environment:
User behavior data, product catalog, inventory, pricing, user profiles.

Actuators:
Web interface (product display), recommendation lists, push notifications, email promotions.

Sensors:
User clicks, browsing history, purchase records, product ratings and reviews.

---

iii) Personal AI Assistant (e.g., Siri, Alexa)

Performance Measure:
Accuracy and relevance of responses, successful task completion, user satisfaction, fast response time.

Environment:
User’s home or work environment, connected devices, online services.

Actuators:
Voice output, screen display, control signals to smart devices, notifications.

Sensors:
Microphone, camera (if applicable), access to calendar, contacts, apps, internet data.

---

iv) Medical Diagnosis System

Performance Measure:
Accuracy of diagnoses, speed of processing, reduction in misdiagnosis, patient safety.

Environment:
Hospitals, clinics, patient data systems, medical literature databases.

Actuators:
Diagnosis reports, alerts, treatment recommendations, medical record updates.

Sensors:
Patient symptoms, test results, medical imaging, electronic health records.

---

v) AI-Based Spam Email Filter

Performance Measure:
High detection rate of spam, minimal false positives and false negatives, fast filtering.

Environment:
Email servers, user inboxes and varying types of incoming emails.

Actuators:
Moving emails to spam folder, flagging suspicious emails, deleting spam messages.

Sensors:
Email content, subject line, sender metadata, embedded links and attachments.

---

vi) Robot Vacuum Cleaner

Performance Measure:
Level of cleanliness achieved, area coverage, efficient use of battery, obstacle avoidance.

Environment:
Indoor space (home/office), furniture layout, floor types, pets, obstacles.

Actuators:
Wheels, rotating brushes, suction motor, dustbin mechanism.

Sensors:
Bump sensors, infrared/cliff sensors, dirt detectors, cameras (in advanced models), gyroscope

RESULTS & DISCUSSION:

The PEAS description for the given AI agents are briefly described in the above experiments every AI agent is evaluated using PEAS description

MAPPING OF PO & PSO:

CO No	PO1	PO3	PO5	PSO1
1	2	2	3	2

ASSESSMENT:

Particulars	Max. Marks	Marks awarded
Preparation	10	
Conduct of Experiment	30	
Results & Discussion	30	
Viva - Voce	20	
Report	10	
Total	**100**	
Evaluator's Signature		

DESIGN OF AN AI AGENT FOR 2D GRID MAZE NAVIGATION USING HEURISTIC SEARCH STRATEGIES

Exp. No.:2

Date :

AIM:

Aim is to implement an Design of an ai agent for 2d grid maze navigation using Heuristic search strategies.

ALGORITHM:

- Initialize the solution matrix with all zeros to track the visited path.
- Start from the initial cell (0, 0) and recursively explore possible directions (down, right, up, left).
- At each step, check if the cell is within bounds, not visited, and not blocked in the maze.
- If a valid move leads to the destination (bottom-right corner), mark the path and return success.
- If no direction leads to the solution, backtrack by unmarking the cell and return failure to try other paths.

CODE AND OUTPUT:

```
SIZE = 5

maze = [
    [0, 1, 0, 1, 1],
    [0, 0, 0, 0, 0],
    [1, 0, 1, 0, 1],
    [0, 0, 1, 0, 0],
    [1, 0, 0, 1, 0]
]
```

```
solution = [[0 for _ in range(SIZE)] for _ in range(SIZE)]

def print_solution():
    for row in solution:
        print('\t'.join(map(str, row)))
    print()

def is_valid(r, c):
    return 0 <= r < SIZE and 0 <= c < SIZE and maze[r][c] == 0 and solution[r][c] == 0

def solve_maze(r, c):
    if r == SIZE - 1 and c == SIZE - 1:
        solution[r][c] = 1
        return True

    if is_valid(r, c):
        solution[r][c] = 1
        if solve_maze(r + 1, c): return True
        if solve_maze(r, c + 1): return True
        if solve_maze(r - 1, c): return True
        if solve_maze(r, c - 1): return True
        solution[r][c] = 0

    return False

if solve_maze(0, 0):
    print_solution()
else:
    print("No solution")
```

```
Solution matrix:
1   0   0   0   0
1   1   1   1   0
0   0   0   1   0
0   0   0   1   1
0   0   0   0   1

Path taken:
(0,0) -> (1,0) -> (1,1) -> (1,2) -> (1,3) -> (2,3) -> (3,3) -> (3,4) -> (4,4)
```

RESULTS & DISCUSSION:

Thus the program to implement an Design of an ai agent for 2d grid maze navigation using Heuristic search strategies to handle input and the output is verified successfully.

MAPPING OF PO & PSO:

CO No	PO1	PO4	PO5	PSO2
1	2	1	3	2

ASSESSMENT:

Particulars	Max. Marks	Marks awarded
Preparation	10	
Conduct of Experiment	30	
Results & Discussion	30	
Viva - Voce	20	
Report	10	
Total	**100**	
Evaluator's Signature		

BUILD AND TRAIN MACHINE LEARNING CLASSIFIER TO CATEGORIZE SALES DATA USING DECISION TREE CLASSIFIER

Exp. No.:3

Date :

AIM:

Aim is to Build and train machine learning classifier to categorize sales data using decision tree classifier

ALGORITHM:

- Load the dataset from the specified CSV file.
- Convert the "Date" column to datetime format.
- Apply one-hot encoding to categorical columns: Branch, City, Product line, and Payment (keeping all categories).
- Map categorical binary fields ("Customer type" and "Gender") to numerical values.
- Define the list of features (columns) in a specific order to ensure consistency during prediction.
- Split the data into input features (X) and target variable (y).
- Split the dataset into training and testing sets using a 67-33 ratio.
- Train a Linear Regression model on the training data.
- Create a function to take user input for all features, including categorical choices.
- Convert the user input into a DataFrame with the same structure and column order as the training data.
- Predict the total sales using the trained model and the user-provided inputs.
- Print the predicted total sales value.

CODE AND OUTPUT:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv('/kaggle/input/supermarket-sales/supermarket_sales - Sheet1.csv')
df['Date'] = pd.to_datetime(df['Date'])

df = pd.get_dummies(df, columns=["Branch", "City", "Product line", "Payment"], drop_first=False)
df['Customer type'] = df['Customer type'].map({'Member': 1, 'Normal': 0})
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})

columns_order = [
    'Customer type', 'Gender', 'Unit price', 'Quantity', 'Tax 5%', 'cogs',
    'gross margin percentage', 'gross income', 'Rating',
    'Branch_B', 'Branch_C',
    'City_Naypyitaw', 'City_Yangon',
    'Product line_Fashion accessories', 'Product line_Food and beverages',
    'Product line_Health and beauty', 'Product line_Home and lifestyle',
    'Product line_Sports and travel',
    'Payment_Credit card', 'Payment_Ewallet'
]

X = df[columns_order]
y = df['Total']

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=88, test_size=0.33)

model = LinearRegression()
model.fit(X_train, y_train)

def get_user_input():
    inputs = {
        'Customer type': int(input("Customer Type (1 for Member, 0 for Normal): ")),
        'Gender': int(input("Gender (1 for Male, 0 for Female): ")),
        'Unit price': float(input("Unit Price: ")),
```

```
        'Quantity': int(input("Quantity: ")),
        'Tax 5%': float(input("Tax (5%): ")),
        'cogs': float(input("Cost of Goods Sold (COGS): ")),
        'gross margin percentage': float(input("Gross Margin Percentage (e.g., 4.76): ")),
        'gross income': float(input("Gross Income: ")),
        'Rating': float(input("Customer Rating (1-10): ")),
        'Branch_B': 0, 'Branch_C': 0,
        'City_Naypyitaw': 0, 'City_Yangon': 0,
        'Product line_Fashion accessories': 0,
        'Product line_Food and beverages': 0,
        'Product line_Health and beauty': 0,
        'Product line_Home and lifestyle': 0,
        'Product line_Sports and travel': 0,
        'Payment_Credit card': 0,
        'Payment_Ewallet': 0
    }

    branch = input("Branch (A, B, or C): ").upper()
    if branch == 'B':
        inputs['Branch_B'] = 1
    elif branch == 'C':
        inputs['Branch_C'] = 1

    city = input("City (Yangon, Naypyitaw, Mandalay): ").capitalize()
    if city == 'Yangon':
        inputs['City_Yangon'] = 1
    elif city == 'Naypyitaw':
        inputs['City_Naypyitaw'] = 1

    product = input("Product Line (Fashion accessories, Food and beverages, Health and beauty, Home and lifestyle,
Sports and travel): ")
    product_col = f"Product line_{product}"
    if product_col in inputs:
        inputs[product_col] = 1

    payment = input("Payment Method (Cash, Ewallet, Credit card): ").capitalize()
    if payment == 'Credit card':
        inputs['Payment_Credit card'] = 1
```

```
    elif payment == 'Ewallet':
        inputs['Payment_Ewallet'] = 1

    return pd.DataFrame([inputs], columns=columns_order)

user_input_df = get_user_input()
predicted_total = model.predict(user_input_df)[0]
print(f"\n Predicted Total Sales: {predicted_total:.2f}")
```

```
Customer Type (1 for Member, 0 for Normal):  1
Gender (1 for Male, 0 for Female):  1
Unit Price:  200
Quantity:  3444
Tax (5%):  33
Cost of Goods Sold (COGS):  2334
Gross Margin Percentage (e.g., 4.76):  4.45
Gross Income:  2000000000000000000
Customer Rating (1-10):  9
Branch (A, B, or C):  B
City (Yangon, Naypyitaw, Mandalay):  Yangon
Product Line (Fashion accessories, Food and beverages, Health and beauty,
Home and lifestyle, Sports and travel):  Sports and travel
Payment Method (Cash, Ewallet, Credit card):  Cash



 Predicted Total Sales: 104477611940300848.00
```

RESULTS & DISCUSSION:

Thus the program to Build and train machine learning classifier to categorize sales data using decision tree classifier is executed and the output is verified successfully.

MAPPING OF PO & PSO:

CO No	PO1	PO2	PO5	PO12
3	1	1	3	2

ASSESSMENT:

Particulars	**Max. Marks**	**Marks awarded**
Preparation	10	
Conduct of Experiment	30	
Results & Discussion	30	
Viva - Voce	20	
Report	10	
Total	**100**	
Evaluator's Signature		

CREATE AND TRAIN A LINEAR REGRESSION MODEL FOR PREDICTING HOUSE PRICES

Exp. No.:4

Date :

AIM:

Aim is to Create and train a linear regression model for predicting house prices

ALGORITHM:

1. Import required libraries for data manipulation, visualization, and machine learning.
2. Load the dataset from an Excel file named **HousePricePrediction.xlsx**.
3. Identify and separate object, integer, and float type columns from the dataset.
4. Create a correlation heatmap of all numerical features for visual analysis.
5. Drop the **Id** column as it's not relevant for prediction.
6. Handle missing values in the **SalePrice** column by replacing them with the mean.
7. Remove rows with any remaining missing values to clean the dataset.
8. Re-identify object (categorical) columns in the cleaned dataset.
9. Apply One-Hot Encoding to convert categorical columns into numerical format.
10. Drop original categorical columns and concatenate the encoded columns to the cleaned dataset.
11. Define input features (X) and the target variable (Y) as **SalePrice**.
12. Split the dataset into training and validation sets (80% training, 20% validation).
13. Initialize and train a **Linear Regression** model on the training set.

14. Predict **SalePrice** on the validation set using the trained model.

15. Calculate and print performance metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score
- Mean Squared Error (MSE)
- Select a sample row from the validation set and use it to predict house price.
- Display the input data and predicted price for the selected test case.
- Show all plots generated (e.g., heatmap)

CODE AND OUTPUT:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import
mean_absolute_error,root_mean_squared_error,r2_score,mean_squared_error
df=pd.read_excel("HousePricePrediction.xlsx")
obj = (df.dtypes == 'object')
object_cols = list(obj[obj].index)
int_ = (df.dtypes == 'int')
num_cols = list(int_[int_].index)
fl = (df.dtypes == 'float')
fl_cols = list(fl[fl].index)
numerical_dataset = df.select_dtypes(include=['number'])
plt.figure(figsize=(12, 6))
sns.heatmap(numerical_dataset.corr(),
cmap = 'BrBG',
fmt = '.2f',
linewidths = 2,
annot = True)
df.drop(['Id'],axis=1,inplace=True)
df['SalePrice'] = df['SalePrice'].fillna(df['SalePrice'].mean())
```

```
new_dataset = df.dropna()
s = (new_dataset.dtypes == 'object')
object_cols = list(s[s].index)
OH_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
OH_cols = pd.DataFrame(OH_encoder.fit_transform(new_dataset[object_cols]))
OH_cols.index = new_dataset.index
OH_cols.columns = OH_encoder.get_feature_names_out()
df_final = new_dataset.drop(object_cols, axis=1)
df_final = pd.concat([df_final, OH_cols], axis=1)
X = df_final.drop(['SalePrice'], axis=1)
Y = df_final['SalePrice']
X_train, X_valid, Y_train, Y_valid = train_test_split(X, Y, train_size=0.8, test_size=0.2,
random_state=0)
model_LR = LinearRegression()
model_LR.fit(X_train, Y_train)
Y_pred = model_LR.predict(X_valid)
print("mae",mean_absolute_error(Y_valid, Y_pred))
print("rmse",root_mean_squared_error(Y_valid, Y_pred))
print("r2",r2_score(Y_valid, Y_pred))
print("mse",mean_squared_error(Y_valid, Y_pred))
test=X_valid.iloc[[0]]
print(X_valid.iloc[[0]])
y_p=model_LR.predict(test)
print("price of the house with tested data :",y_p[0])
plt.show()
```

OUTPUT:

```
rmse 41138.55571665912
r2 0.3741422704574483
mse 1692380766.4526668
      MSSubClass  LotArea  ...  Exterior1st_Wd Sdng  Exterior1st_WdShing
1728          60    10274  ...                  0.0                  0.0

[1 rows x 37 columns]
price of the house with tested data : 191650.15923449374

Process finished with exit code 0
```

RESULTS & DISCUSSION:

Thus the program to Create and train a linear regression model for predicting house prices is executed and the output is verified successfully.

MAPPING OF PO & PSO:

CO No	PO1	PO2	PO3	PO5	PSO1
4	1	1	2	3	2

ASSESSMENT:

Particulars	Max. Marks	Marks awarded
Preparation	10	
Conduct of Experiment	30	
Results & Discussion	30	
Viva - Voce	20	
Report	10	
Total	**100**	
Evaluator's Signature		

DESIGN AND IMPLEMENT OF A REINFORCEMENT LEARNING BASED TREASURE COLLECTION AGENT IN A 6*6 GRID ENVIRONMENT

Exp. No.: 5
Date:

AIM:

Aim is to Design and implement of a reinforcement learning based treasure collection agent in a 6*6 grid environment

ALGORITHM:

- Import numpy and random libraries.
- Set grid size to 6.
- Define possible actions: up, down, left, right.
- Create a dictionary to map actions to indices.
- Set learning rate to 0.1.
- Set discount factor to 0.9.
- Set exploration rate to 0.2.
- Set number of training episodes to 500.
- Initialize a Q-table with zeros for all grid positions and actions.
- Create a function to place the treasure randomly.
- Create a function to move the agent based on the chosen action.
- Create a function to give a reward: +10 for finding treasure, -1 otherwise.
- Loop over episodes.
- Randomly choose a starting state and place the treasure.
- Repeat until the agent finds the treasure:
    a. Choose a random action with probability epsilon or the best action from Q-table.

b. Move to the next state using the chosen action.
c. Get reward for the new state.
d. Update Q-table using the Q-learning formula.
e. Set current state to the new state.

- Stop when treasure is found.
- Define a test function to check agent behavior after training.
- Start at position (0, 0) and generate a new treasure.
- Follow best action from Q-table until the treasure is reached.
- Save the path taken by the agent.
- Print the treasure location.
- Print the path followed by the agent.
- Create a grid to represent the final environment.
- Mark treasure with T and agent's path with A.
- Print the final environment grid

CODE:

```
import numpy as np, random, matplotlib.pyplot as plt

grid = [
['S', '.', '.', '.', '.', '.'],
['X', 'X', '.', '.', '.', '.'],
['.', '.', 'X', '.', '.', '.'],
['.', '.', '.', 'X', '.', '.'],
['.', '.', '.', '.', 'X', '.'],
['.', '.', '.', '.', '.', 'T']
]

start = [0, 0]
treasure = [5, 5]
obstacles = [[r, c] for r in range(6) for c in range(6) if grid[r][c] == 'X']
Q = {}
actions = [0, 1, 2, 3]
alpha, gamma, eps = 0.1, 0.9, 1.0
```

```
def step(pos, a):
r, c = pos
if a == 0 and r > 0: r -= 1
elif a == 1 and r < 5: r += 1
elif a == 2 and c > 0: c -= 1
elif a == 3 and c < 5: c += 1
nxt = [r, c]
if nxt in obstacles: return pos, -5, False
done = nxt == treasure
reward = 10 if done else -1
return nxt, reward, done

for _ in range(1000):
s = start[:]
for _ in range(50):
t = tuple(s)
Q.setdefault(t, [0]*4)
a = random.choice(actions) if random.random() < eps else np.argmax(Q[t])
ns, r, d = step(s, a)
nt = tuple(ns)
Q.setdefault(nt, [0]*4)
Q[t][a] += alpha * (r + gamma * max(Q[nt]) - Q[t][a])
s = ns
if d: break
eps = max(0.1, eps * 0.995)

s, path = start[:], [start[:]]
for _ in range(30):
t = tuple(s)
if t not in Q: break
a = np.argmax(Q[t])
s, _, d = step(s, a)
path.append(s[:])
if d: break

print("Input Grid:")
for row in grid: print(' '.join(row))
print("\nPath Taken:")
print(' -> '.join([f"({r},{c})" for r, c in path]))

g = [['.' for _ in range(6)] for _ in range(6)]
for r, c in path: g[r][c] = 'A'
for r, c in obstacles: g[r][c] = 'X'
```

```
g[start[0]][start[1]] = 'S'
g[treasure[0]][treasure[1]] = 'T'

plt.imshow(np.ones((6, 6)), cmap='gray_r')
for r in range(6):
for c in range(6):
ch = g[r][c]
plt.text(c, r, ch, ha='center', va='center', fontsize=12,
color={'A':'red', 'X':'blue', 'T':'green', 'S':'orange'}.get(ch, 'black'))
plt.axis('off'), plt.title("Q-learning Agent Path"), plt.show()
```

OUTPUT:

```
Input Grid:
S . . . . .
X X . . . .
. . X . . .
. . . X . .
. . . . X .
. . . . . T

Path Taken:
(0,0) -> (0,1) -> (0,2) -> (0,3) -> (0,4) -> (0,5) -> (1,5) -> (2,5) -> (3,5) -> (4,5) -> (5,5)
```

```
((np.float64(-0.5), np.float64(5.5), np.float64(5.5), np.float64(-0.5)),
 Text(0.5, 1.0, 'Q-learning Agent Path'),
 None)
```

RESULTS & DISCUSSION:

Thus the program to Design and implement of a reinforcement learning based treasure collection agent in a 6*6 grid environment is executed and the output is verified successfully.

MAPPING OF PO & PSO:

CO No	PO1	PO2	PO3	PO5
4	2	3	2	3

ASSESSMENT:

Particulars	Max. Marks	Marks awarded
Preparation	10	
Conduct of Experiment	30	
Results & Discussion	30	
Viva - Voce	20	
Report	10	
Total	**100**	
Evaluator's Signature		

DESIGN AND IMPLEMENT NLP BASED NAIVE BAYES CLASSIFIER FOR CLASSIFICATION OF SPAM EMAIL DETECTION

Exp. No.:6

Date :

AIM:

Aim is to create Design and implement nlp based naive bayes classifier for classification of spam email detection

ALGORITHM:

- Install Express, Mongoose, CORS, and configure environment variables for MongoDB connectivity using .env.
- Connect to MongoDB with Mongoose, handling errors and confirming connection success.
- Define a Mongoose product schema with mandatory name and price fields.
- Initialize an Express server, integrating JSON parsing and CORS middleware.
- Setup Express routes for CRUD operations linked to controller functions.
- Implement controller functions for database interactions, error handling, and response management.
- Launch the server, test endpoints with Postman to ensure functionality..

CODE AND OUTPUT:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

df = pd.read_csv("combined_data.csv")
print(df["label"].value_counts())

texts = df['text'].astype(str).tolist()
labels = df['label'].astype(int).tolist()

vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(texts).toarray()
```

```
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2, random_state=42)

clf = GaussianNB()
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))

i = input("Enter the email to do classification: ")
user_vector = vectorizer.transform([i]).toarray()
if clf.predict(user_vector)[0] == 1:
    print("spam")
else:
    print("not-spam")
```

OUTPUT:

```
/Users/arjunaaditiyaas/PycharmProjects/PythonProject/.venv/bin/python /Users/arjunaaditiyaas/PycharmProjects/PythonProject/SpamClassifier.py
label
1    43910
0    39538
Name: count, dtype: int64
              precision    recall  f1-score   support

           0       0.96      0.97      0.97      7938
           1       0.97      0.97      0.97      8752

    accuracy                           0.97     16690
   macro avg       0.97      0.97      0.97     16690
weighted avg       0.97      0.97      0.97     16690

Enter the email to do classification:
hello you won 10000000 dollars click the link to claim it
spam

Process finished with exit code 0
```

RESULTS AND DISCUSSION:

Thus the program to Design and implement nlp based naive bayes classifier for classification of spam email detection is executed and the output is verified successfully.

MAPPING OF PO & PSO:

CO No	PO1	PO2	PO3	PO5
4	1	2	2	3

ASSESSMENT:

Particulars	Max. Marks	Marks awarded
Preparation	10	
Conduct of Experiment	30	
Results & Discussion	30	
Viva - Voce	20	
Report	10	
Total	**100**	
Evaluator's Signature		